

Hashing Methods Benchmark Report

Overview

This report summarizes the benchmarking of three hash table collision resolution strategies: Separate Chaining with Balanced BST, Linear Probing, and Double Hashing. Experiments used two distinct string hash functions (djb2 and Polynomial rolling hash), evaluating their effectiveness across multiple load factors.

Hash Functions

- djb2 hash: Efficient and widely used, starting with an initial hash of 5381 and using multiplication by 33 per character.
- Polynomial rolling hash: Uses modular arithmetic with a base (commonly 31) for each character position.

djb2: hash = 5381; for each c: hash = ((hash << 5) + hash) + c; modulo table size
poly: hash = $\sum_{i=0}^{n-1} (s[i] - 'a' + 1) * p^i \pmod{\text{table size}}$

Experimental Protocol

- Table size is set to a prime number for better key distribution.
- Random lowercase words (length 5-10) are generated and used as keys.
- For each load factor, results are collected before and after deletions
- Metrics: #collisions, average search times, average probe count.

Results

Load Factor: 0.4

Hash1 = djb2

Method	Hash1	Hash2	#Coll	AvgTime (before)	Probes (before)	AvgTime (after)	Probes (after)
Separate Chaining	djb2	–	670	155.45 ns	N/A	131.50 ns	N/A
Linear Probing	djb2	–	1242	166.89 ns	1.31	162.60 ns	2.17
Double Hashing	djb2	poly	1841	174.91 ns	1.46	256.50 ns	2.53

Hash1 = poly

Method	Hash1	Hash2	#Coll	AvgTime (before)	Probes (before)	AvgTime (after)	Probes (after)
Separate Chaining	poly	–	719	596.08 ns	N/A	397.40 ns	N/A
Linear Probing	poly	–	1331	376.19 ns	1.33	566.20 ns	2.29
Double Hashing	poly	djb2	2004	491.30 ns	1.50	488.60 ns	2.45

Load Factor: 0.5

Hash1 = djb2

Method	Hash1	Hash2	#Coll	AvgTime (before)	Probes (before)	AvgTime (after)	Probes (after)
Separate Chaining	djb2	–	1021	276.57 ns	N/A	281.50 ns	N/A
Linear Probing	djb2	–	2442	203.74 ns	1.49	410.40 ns	2.75
Double Hashing	djb2	poly	3030	209.77 ns	1.61	293.10 ns	2.73

Hash1 = poly

Method	Hash1	Hash2	#Coll	AvgTime (before)	Probes (before)	AvgTime (after)	Probes (after)
Separate Chaining	poly	–	1069	485.67 ns	N/A	443.30 ns	N/A
Linear Probing	poly	–	2337	530.34 ns	1.47	511.10 ns	2.71
Double Hashing	poly	djb2	3243	483.41 ns	1.65	497.20 ns	2.77

Load Factor: 0.6

Hash1 = djb2

Method	Hash1	Hash2	#Coll	AvgTime (before)	Probes (before)	AvgTime (after)	Probes (after)
Separate Chaining	djb2	–	1450	322.72 ns	N/A	322.70 ns	N/A
Linear Probing	djb2	–	4353	380.60 ns	1.73	677.00 ns	3.42
Double Hashing	djb2	poly	4784	368.05 ns	1.80	535.00 ns	3.04

Hash1 = poly

Method	Hash1	Hash2	#Coll	AvgTime (before)	Probes (before)	AvgTime (after)	Probes (after)
Separate Chaining	poly	–	1463	619.84 ns	N/A	520.10 ns	N/A
Linear Probing	poly	–	4203	402.83 ns	1.70	497.00 ns	3.61
Double Hashing	poly	djb2	4949	407.88 ns	1.82	488.80 ns	3.06

Load Factor: 0.7

Hash1 = djb2

Method	Hash1	Hash2	#Coll	AvgTime (before)	Probes (before)	AvgTime (after)	Probes (after)
Separate Chaining	djb2	–	1921	425.37 ns	N/A	352.90 ns	N/A
Linear Probing	djb2	–	7603	411.34 ns	2.09	584.10 ns	4.66
Double Hashing	djb2	poly	7282	422.17 ns	2.04	749.00 ns	3.47

Hash1 = poly

Method	Hash1	Hash2	#Coll	AvgTime (before)	Probes (before)	AvgTime (after)	Probes (after)
Separate Chaining	poly	–	1947	401.20 ns	N/A	412.80 ns	N/A
Linear Probing	poly	–	7707	434.42 ns	2.10	655.70 ns	5.27
Double Hashing	poly	djb2	7498	397.10 ns	2.07	638.60 ns	3.46

Load Factor: 0.8

Hash1 = djb2

Method	Hash1	Hash2	#Coll	AvgTime (before)	Probes (before)	AvgTime (after)	Probes (after)
Separate Chaining	djb2	–	2463	415.58 ns	N/A	350.10 ns	N/A
Linear Probing	djb2	–	14385	499.59 ns	2.80	937.00 ns	8.69
Double Hashing	djb2	poly	11217	530.97 ns	2.40	712.80 ns	4.49

Hash1 = poly

Method	Hash1	Hash2	#Coll	AvgTime (before)	Probes (before)	AvgTime (after)	Probes (after)
Separate Chaining	poly	–	2452	384.51 ns	N/A	327.50 ns	N/A
Linear Probing	poly	–	17164	513.94 ns	3.14	1050.60 ns	10.05
Double Hashing	poly	djb2	11168	452.14 ns	2.40	1009.50 ns	4.62

Load Factor: 0.9

Hash1 = djb2

Method	Hash1	Hash2	#Coll	AvgTime (before)	Probes (before)	AvgTime (after)	Probes (after)
Separate Chaining	djb2	–	3044	430.07 ns	N/A	414.70 ns	N/A
Linear Probing	djb2	–	34586	678.87 ns	4.84	2892.70 ns	27.46
Double Hashing	djb2	poly	17749	699.10 ns	2.97	1113.00 ns	7.15

Hash1 = poly

Method	Hash1	Hash2	#Coll	AvgTime (before)	Probes (before)	AvgTime (after)	Probes (after)
Separate Chaining	poly	–	3034	438.81 ns	N/A	411.80 ns	N/A
Linear Probing	poly	–	39755	794.08 ns	5.41	3244.60 ns	36.39
Double Hashing	poly	djb2	17939	572.52 ns	2.99	885.90 ns	7.27

Discussion

As load factor rises, separate chaining stays efficient while linear probing and double hashing see increased collision rates and probe counts. Double hashing outperforms linear probing at high loads but chaining is most stable.

Conclusion

Among the tested approaches, separate chaining with BST offers the most predictable performance. Double hashing is a strong compromise, especially at moderate loads. The djb2 and polynomial hashes both provide strong key distribution. Method and hash function choice should depend on application constraints.